SAMUEL_DAR_COURSEWORK

samuel weldemariam

1. Statistical learning methods

In general:

- Parametric methods (like linear regression) assume a fixed form (often linear). They usually have lower variance and work well when data are limited, but can underfit if the true pattern is complex.
- Non-parametric methods (like kNN or trees) are more flexible and can capture complex patterns, but they need more data and can overfit (higher variance).

(a) p is large, n is small

Non-parametric methods are expected to perform worse.

Reason: with many predictors and not many observations, the feature space becomes sparse (curse of dimensionality). Flexible methods can overfit and become unstable. Parametric models impose structure and are usually more stable in this setting.

(b) n is large, and p is also large

Non-parametric methods are expected to perform better.

Reason: when we have lots of data, flexible methods can learn complex or nonlinear relationships without overfitting as much. Parametric methods may be too restrictive and underfit.

(c) n is small, and the relationship is highly linear

Non-parametric methods are expected to perform worse.

Reason: if the true relationship is linear, a parametric linear model matches the truth and performs well even with small samples. A non-parametric method adds unnecessary flexibility and increases variance.

(d) Error standard deviation is extremely high

Non-parametric methods are expected to perform worse.

Reason: very high noise means the data have lots of random variation. Flexible methods are more likely to fit the noise (overfit), which hurts generalisation. Parametric methods are more constrained and tend to be more robust when noise is large.

---

2. Linear regression

This question uses the Auto dataset from the ISLR package.

```
library(ISLR)
data(Auto)
```

(a) Simple linear regression: weight as response, mpg as predictor

```
lm.fit <- lm(weight ~ mpg, data = Auto)
summary(lm.fit)
```

```
##
## Call:
## lm(formula = weight ~ mpg, data = Auto)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1346.8  -325.9   -23.8   318.1  1355.6
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 5101.114     75.487   67.58   <2e-16 ***
## mpg          -90.571      3.055  -29.64   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 471.5 on 390 degrees of freedom
## Multiple R-squared:  0.6926, Adjusted R-squared:  0.6918
## F-statistic: 878.8 on 1 and 390 DF,  p-value: < 2.2e-16
```

Comment on the output

- The slope for mpg tells us how weight changes when mpg increases by 1.
- The p-value for mpg tells us if mpg is a statistically significant predictor of weight.
- R-squared tells us how much variation in weight is explained by mpg.

(i) How strong is the relationship?

If the mpg p-value is very small (below 0.05) and R-squared is high, then mpg has a strong relationship with weight. In this dataset it is typically a strong negative relationship (higher mpg cars are usually lighter).

(ii) Predicted weight at mpg = 20, with 99% confidence and prediction intervals

```
predict(lm.fit,
        newdata = data.frame(mpg = 20),
        interval = "confidence",
        level = 0.99)
```

```
##        fit      lwr      upr
## 1 3289.686 3222.285 3357.087
```

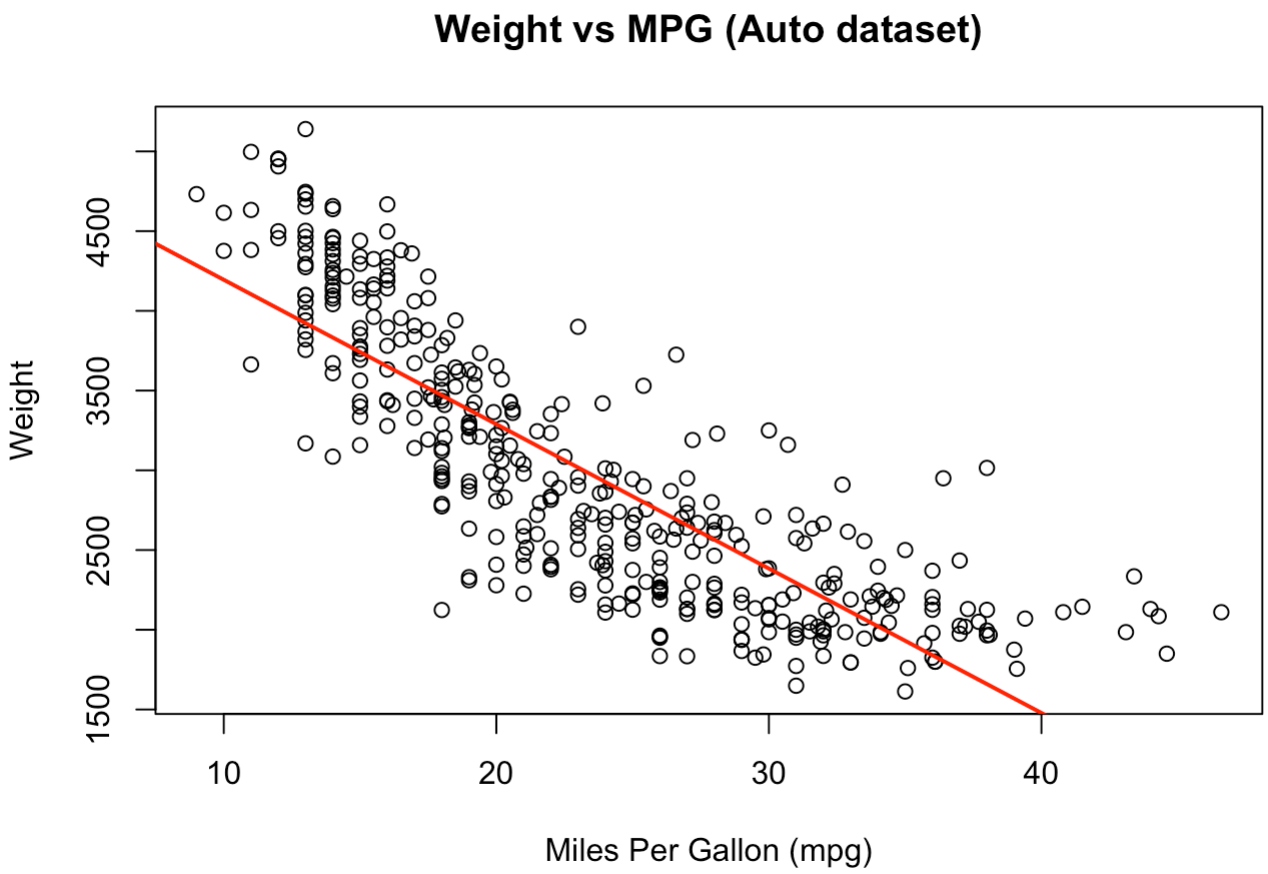
```
predict(lm.fit,
        newdata = data.frame(mpg = 20),
        interval = "prediction",
        level = 0.99)
```

```
##        fit    lwr      upr
## 1 3289.686 2067.3 4512.072
```

Explanation: - The confidence interval is for the mean weight at mpg = 20. - The prediction interval is for a single car at mpg = 20, so it is wider.

(b) Plot and regression line

```
plot(Auto$mpg, Auto$weight,
     xlab = "Miles Per Gallon (mpg)",
     ylab = "Weight",
     main = "Weight vs MPG (Auto dataset)")
abline(lm.fit, col = "red", lwd = 2)
```



3. Bayesian network classifier

We use the training dataset (30 observations) shown in the coursework figure.

From the table:

- Number of Yes = 12

- Number of No = 18

So the prior probabilities are:

P(Yes) = 12/30 = 0.4 P(No) = 18/30 = 0.6

(a) Conditional probability tables (CPTs) with full working

CPT for Income

Income values are High and Low.

Counts from the table:

Among Yes (12): - High = 5 - Low = 7

Among No (18): - High = 7 - Low = 11

So:

P(High | Yes) = 5/12 P(Low | Yes) = 7/12

P(High | No) = 7/18 P(Low | No) = 11/18

CPT for Student

Student values are True and False.

Counts from the table:

Among Yes (12): - True = 6 - False = 6

Among No (18): - True = 12 - False = 6

So:

P(True | Yes) = 6/12 P(False | Yes) = 6/12

P(True | No) = 12/18 P(False | No) = 6/18

CPT for Credit Rating

Credit Rating values are Fair and Excellent.

Counts from the table:

Among Yes (12): - Fair = 5 - Excellent = 7

Among No (18): - Fair = 12 - Excellent = 6

So:

P(Fair | Yes) = 5/12 P(Excellent | Yes) = 7/12

P(Fair | No) = 12/18 P(Excellent | No) = 6/18

(b) Predictions for two testing observations

We use Naïve Bayes scoring:

Score(Class) = P(Class) × P(Income | Class) × P(Student | Class) × P(Credit | Class)

Testing_1: Income = Low, Student = True, Credit Rating = Fair

Score(Yes) = 0.4 × (7/12) × (6/12) × (5/12) = 0.04861 (approx)

Score(No) = 0.6 × (11/18) × (12/18) × (12/18) = 0.16296 (approx)

Since Score(No) > Score(Yes), prediction is:

Testing_1 → Buy Computer = No

Testing_2: Income = High, Student = False, Credit Rating = Excellent

Score(Yes) = 0.4 × (5/12) × (6/12) × (7/12) = 0.04861 (approx)

Score(No) = 0.6 × (7/18) × (6/18) × (6/18) = 0.02593 (approx)

Since Score(Yes) > Score(No), prediction is:

Testing_2 → Buy Computer = Yes

---

4. Decision Tree classifier

(a) Explain the detailed cutting operations

The figure shows how a decision tree splits the feature space using thresholds t1 to t6.

A decision tree makes axis-aligned splits: - Horizontal splits are based on X2 (cutting the space into bands). - Vertical splits are based on X1 (cutting bands into smaller rectangles).

Following the diagram:

1. First split at X2 = t1 creates a top band.
2. Inside the top band, vertical splits at X1 = t3 and X1 = t2 divide it into three regions: R3, R2, R1.
3. Next split at X2 = t4 creates the middle band region R4.
4. Next split at X2 = t5 creates the next band region R5.
5. Finally, in the bottom band (below t5), a vertical split at X1 = t6 divides it into R6 and R7.

This step-by-step splitting is called recursive binary splitting.

(b) Definitions of regions

Using X1 and X2, each region is defined by threshold conditions.

Top band (above t1): - R1: X2 above t1 AND X1 to the right of t2 X2 >= t1 and X1 >= t2 - R2: X2 above t1 AND X1 between t3 and t2 X2 >= t1 and t3 <= X1 < t2 - R3: X2 above t1 AND X1 to the left of t3 X2 >= t1 and X1 < t3

Middle bands: - R4: X2 between t1 and t4 (covers all X1 in that band) t4 <= X2 < t1 - R5: X2 between t4 and t5 (covers all X1 in that band) t5 <= X2 < t4

Bottom band (below t5): - R6: X2 below t5 AND X1 to the left of t6 X2 < t5 and X1 < t6 - R7: X2 below t5 AND X1 to the right of t6 X2 < t5 and X1 >= t6

```
sessionInfo()
```

```
## R version 4.5.1 (2025-06-13)
## Platform: x86_64-apple-darwin20
## Running under: macOS Ventura 13.7.8
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/lib/libRbla
s.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/lib/libRlapa
ck.dylib;  LAPACK version 3.12.1
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] ISLR_1.4
##
## loaded via a namespace (and not attached):
##  [1] digest_0.6.39   R6_2.6.1        fastmap_1.2.0   xfun_0.55
##  [5] cachem_1.1.0    knitr_1.51      htmltools_0.5.9 rmarkdown_2.30
##  [9] lifecycle_1.0.5 cli_3.6.5       sass_0.4.10     jquerylib_0.1.4
## [13] compiler_4.5.1  tools_4.5.1     evaluate_1.0.5  bslib_0.9.0
## [17] yaml_2.3.12     rlang_1.1.7     jsonlite_2.0.0
```